Polimorfismo y sobrecarga

Estos conceptos suelen confundirse entre sí por su funcionalidad parecida, pero su principio teórico es diferente. Por un lado, cuando hablamos de sobrecarga nos referimos a un conjunto de funciones o métodos que varían sus argumentos, pero mantienen el mismo nombre. Por otro lado, el polimorfismo es un pilar de POO y se vincula directamente al aplicar herencia permitiendo la sobre escritura de métodos y por lo tanto distintos comportamientos según de donde se invoque.

Sobrecarga en C++

La sobrecarga en C++ es una característica que permite definir múltiples funciones o métodos con el mismo nombre, pero con diferentes listas de parámetros. Esta capacidad se conoce como sobrecarga de funciones o sobrecarga de operadores, dependiendo de su aplicación.

La sobrecarga de funciones se da cuando varias funciones comparten el mismo nombre, pero difieren en el tipo o número de parámetros. El compilador distingue cuál función invocar basándose en el número y tipo de argumentos pasados en la llamada.

Reglas para la Sobrecarga de Funciones:

- Número de parámetros: Se puede cambiar la cantidad de parámetros en las distintas versiones de la función.
- Tipos de parámetros: Puedes cambiar el tipo de los parámetros.
- Orden de parámetros: Si las funciones tienen el mismo número de parámetros pero de tipos diferentes, también se considera una sobrecarga válida.
- Tipo de retorno: No se considera sobrecarga solo por cambiar el tipo de retorno. El compilador no distingue entre funciones solo por sus tipos de retorno.
-

```
class persona
{
private:
    int edad;
    string nombre;
public:
    persona(){}
    void set_datos(int);
    void set_datos(string);
    void set_datos(int,string);
};
persona::set_datos(int n_edad){this->edad=n_edad;}
persona::set_datos(string n_nombre){this->nombre=n_nombre;}
persona::set_datos(int n_edad,string n_nombre)
{
    this->edad=n_edad;
    this->nombre=n_nombre;
}
```

En el ejemplo anterior contamos con una clase que se llama persona, esta clase contiene tres métodos sobrecargados, son tres métodos con el mismo nombre, pero difieren en la cantidad y/o tipo de argumentos.

Ahora cuando instanciamos un objeto persona en el main y accedemos a las propiedades del objeto buscando el método set_datos, nos encontramos con el siguiente menú que nos indica que existe una sobrecarga:

```
int main()
{
    persona p;
              void persona::set_datos(int)
                (1/3)
    p.set_datos()
}
```

Según el tipo de dato a pasar en el argumento de la función es el método que va a invocar el compilador. Indirectamente ya vimos esto cuando creamos una clase con un constructor vacío y con un constructor con argumentos, para ese caso lo que estamos aplicando es sobrecarga de constructores.

Sobrecarga de operadores

Existe otro tipo de sobrecarga y es la de operadores, herramienta que nos provee C++ y permite re definir el comportamiento de diversos operadores ante un objeto. Por ejemplo, podemos sobrecargar el operador suma para la clase persona, y poder sumar dos objetos persona y que nos devuelva un resultado; que puede ser otra persona, otro objeto o cualquier otro valor que definamos. Los operadores que se pueden sobrecargar son los matemáticos (+ - * /) y la mayoría de los símbolos (> < ==), etc. Veamos un ejemplo de sobrecarga de suma en persona:

```
class persona
{
private:
    int edad;
    string nombre;
public:
    persona(int edad,string nombre){
    this->edad=edad;
    this->nombre=nombre;
    }
    persona operator +(persona);
};

persona persona::operator+(persona p)
{
    return persona(p.edad+this->edad,p.nombre);
}
```

En el main aplicamos:

```
int main()
{
    persona p(23,"Fabian");
    persona p2(28,"Laura");
    persona p3=p+p2;
}
```

Polimorfismo

La funcionalidad del polimorfismo está directamente vinculada a la herencia. Decimos que una función es polimórfica porque cambia su comportamiento según desde donde se invoque. Por ejemplo, seguimos con la clase persona la cual tiene un método mostrar datos, y aplicando polimorfismo hacemos que esta función se reescriba en las clases derivadas de tal manera que también exista en estas clases derivadas un método mostrar datos con el mismo nombre pero que muestre los datos de la clase deriva y puede también mostrar los de la clase base si es que lo necesitamos.

```
class persona
{
private:
    int edad;
    string nombre;
public:
    persona(int edad,string nombre){
    this->edad=edad;    this->nombre=nombre;    }
    void mostrarDatos();
};
```

```
void persona::mostrarDatos()
{
    cout<<"Edad: "<<this->edad<<endl;
    cout<<"Nombre: "<<this->nombre<<endl;
}
```

A la clase base le creamos el método mostrar datos y luego creamos una clase derivada con el mismo método.

```
class estudiante:public persona
{
private:
    int legajo;
public:
    estudiante(int,int,string);
    void mostrarDatos();
};
```

```
estudiante::estudiante(int legajo,int edad,string nombre)
    :persona(edad,nombre)
    {
        this->legajo=legajo;
    }
void estudiante::mostrarDatos()
{
    persona::mostrarDatos();
    cout<<"Legajo: "<<this->legajo<<endl;
}
```

En el main ocurre lo siguiente:

```
int main()
{
    estudiante el(4,16,"Leo");
    el.mostrarDatos();cout<<endl;
    el.persona::mostrarDatos();cout<<endl;

    persona *p=new estudiante(5,12,"Lucas");
    p->mostrarDatos();
}
```

```
Edad: 16
Nombre: Leo
Legajo: 4

Edad: 16
Nombre: Leo

Edad: 12
Nombre: Lucas
```

Para el caso del objeto e1, invocamos el método de la clase derivada y luego el de la clase base. Ahora para el objeto p, instanciado en estudiante, la función mostrar datos invoca al de la clase base, aunque esté instanciado en estudiante. Podemos forzar el código de tal manera que al tener un objeto persona instanciado en estudiante se invoque el método de estudiante, para esto, debemos modificar en la clase base el método mostrarDatos agregando la palabra virtual:

```
class persona
{
private:
    int edad;
    string nombre;
public:
    persona(int edad,string nombre){
    this->edad=edad;    this->nombre=nombre;    }
    virtual void mostrarDatos();
};
```

Con este pequeño cambio estamos forzando a invocar el método de la clase derivada cuando un objeto base apunta a una clase derivada. Ahora en consola vemos:

```
Edad: 16
Nombre: Leo
Legajo: 4

Edad: 16
Nombre: Leo

Edad: 12
Nombre: Lucas
Legajo: 5
```

El objeto persona con el nombre de lucas invoca mostrarDatos de estudiante.